2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

SparseDrive: End-to-End Autonomous Driving via Sparse Scene Representation

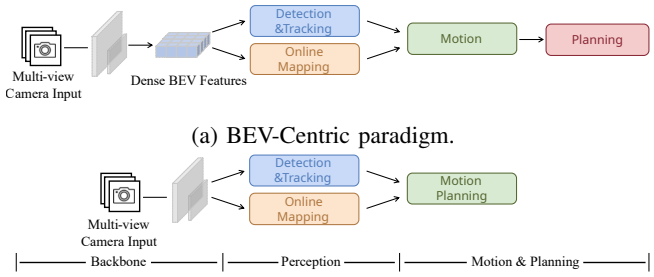
Wenchao Sun[1,2], Xuewu Lin[2], Yining Shi[1], Chuang Zhang[1], Haoran Wu[1] and Sifa Zheng[1]

***Abstract*— The well-established modular autonomous driving system is decoupled into different standalone tasks, e.g. perception, prediction and planning, suffering from information loss and error accumulation across modules. In contrast, end-to-end paradigms unify multi-tasks into a fully differentiable framework, allowing for optimization in a planning-oriented spirit. Despite the great potential of end-to-end paradigms, both the performance and efficiency of existing methods are not satisfactory, particularly in terms of planning safety. We attribute this to the computationally expensive BEV (bird's eye view) features and the straightforward design for prediction and planning. To this end, we explore the sparse representation and review the task design for end-to-end autonomous driving, proposing a new paradigm named SparseDrive. Concretely, SparseDrive consists of a symmetric sparse perception module and a parallel motion planner. The sparse perception module unifies detection, tracking and online mapping with a symmetric model architecture, learning a fully sparse representation of the driving scene. For motion prediction and planning, we review the great similarity between these two tasks, leading to a parallel design for motion planner. Based on this parallel design, which models planning as a multi-modal problem, we propose a hierarchical planning selection strategy, which incorporates a collision-aware rescore module, to select a rational and safe trajectory as the final planning output. With such effective designs, SparseDrive surpasses previous state-of-the-arts by a large margin in performance of all tasks, while achieving much higher training and inference efficiency.**

I. Introduction

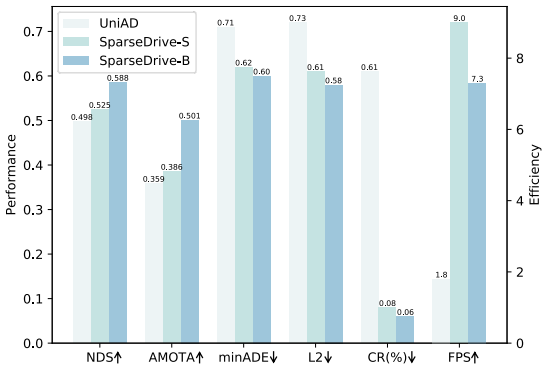
Traditional autonomous driving system is characterized as modular tasks in sequential order. While advantageous in interpretation and error tracking, it inevitably leads to information loss and accumulative errors across successive modules, thereby limiting the optimal performance potential of the system.

Recently, an end-to-end driving paradigm emerged as a promising research direction. This paradigm integrates all tasks into one holistic model, and can be optimized toward the ultimate pursuit for planning. However, the existing methods[1], [2] are not satisfactory in terms of performance and efficiency. On one hand, previous methods rely on computationally expensive BEV features. On the other hand, the straightforward design for prediction and planning limits the model performance. We summarize previous methods as BEV-Centric paradigm in Fig. 1a.



(a) BEV-Centric paradigm.

(b) Sparse-Centric paradigm.



(c) Comparison between our method and previous SOTA[1].

Fig. 1: The comparison of various end-to-end paradigms. (a) The BEV-Centric paradigm. (b) The proposed Sparse-Centric paradigm. (c) Performance and efficiency comparison between (a) and (b).

To fully leverage the potential of end-to-end paradigm, we review the task design of existing methods, and argue that three main parallels shared between motion prediction and planning are neglected as follows: (1) Aiming at predicting future trajectories of surrounding agents and ego vehicle, both motion prediction and planning should consider the high-order and bidirectional interactions among road agents. However, previous methods typically adopt a sequential design for motion prediction and planning, ignoring the impact of ego vehicle on surrounding agents. (2) Accurate prediction for future trajectories requires semantic information for scene understanding, and geometric information to predict future movement of agents, which is applicable to both motion prediction and planning. While these information are extracted in upstream perception tasks for surrounding agents, it is overlooked for ego vehicle. (3) Both motion prediction and

*This work was supported by the National Natural Science Foundation of China under Grant 52221005 and Tsinghua-Toyota Joint Research Institute Inter-disciplinary Program.

[1]School of Vehicle and Mobility, Tsinghua University, China, {swc21, syn21, zhch20, wuhr19}@mails.tsinghua.edu.cn, zsf@mail.tsinghua.edu.cn.
[2]Horizon, China, xuewu.lin@horizon.ai

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11128800

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.

planning are multi-modal problems with inherent uncertainty, but previous methods only predict deterministic trajectory for planning.

To this end, we propose SparseDrive, a Sparse-Centric paradigm as shown in Fig. 1b. Specifically, SparseDrive is composed of a symmetric sparse perception module and a parallel motion planner. With the decoupled instance feature and geometric anchor as complete representation of one instance (a dynamic road agent or a static map element), **Symmetric Sparse Perception** unifies detection, tracking and online mapping tasks with a symmetric model architecture, learning a fully sparse scene representation. In **Parallel Motion Planner**, a semantic-and-geometric-aware ego instance is first obtained from ego instance initialization module. With the ego instance and surrounding agent instances from sparse perception, motion prediction and planning are conducted simultaneously to get multi-modal trajectories for all road agents. To ensure the rationality and safety for planning, a hierarchical planning selection strategy that incorporating a collision-aware rescore module is applied to select the final planning trajectory from multi-modal trajectory proposals.

With above effective designs, SparseDrive unleashes the great potential of end-to-end autonomous driving, as shown in Fig. 1c. Without bells and whistles, our base model, SparseDrive-B, greatly reduces the average L2 error by 19.4% (0.58m vs. 0.72m) and collision rate by 71.4% (0.06% vs. 0.21%). Compared with previous SOTA (state-of-the-art) method UniAD[1], our small model, SparseDrive-S achieves superior performance among all tasks, while running 7.2× faster for training (20 h vs. 144 h) and 5.0× faster for inference (9.0 FPS vs. 1.8 FPS).

The main contribution of our work are summarized as follows:

- We explore the sparse scene representation for end-to-end autonomous driving and propose a Sparse-Centric paradigm named SparseDrive, which unifies multiple tasks with sparse instance representation.
- We revise the great similarity shared between motion prediction and planning, correspondingly leading to a parallel design for motion planner. We further propose a hierarchical planning selection strategy incorporating a collision-aware rescore module to boost the planning performance.
- SparseDrive surpasses previous SOTAs on both open- and closed-loop benchmarks, especially in safety-related metric, while keeping much higher training and inference efficiency.

II. Related Work

A. Multi-view 3D Perception

Multi-view 3D perception is a prerequisite for the safety of autonomous driving system. LSS[3] utilizes depth estimation to lift image features to 3D space and splats to BEV plane for segmentation. Follow-up works apply lift-splat operation to the field of 3D detection, and have made significant improvement in accuracy[4], [5], [6] and efficiency[7], [8]. Some works[9], [10] predefine a set of BEV queries and project them to perspective view for feature sampling. Another line of research removes the dependency for dense BEV features. PETR series[11], [12], [13] introduce 3D positional encoding and global attention to learn view transformation implicitly. Sparse4D series[14], [15], [16] set explicit anchors in 3D space, projecting them to image view to aggregate local features and refine anchors in an iterative fashion. Beyond basic detection task, some works extend to multi-object tracking[16], [17] and online mapping[18], [19], [20] tasks for better environment understanding.

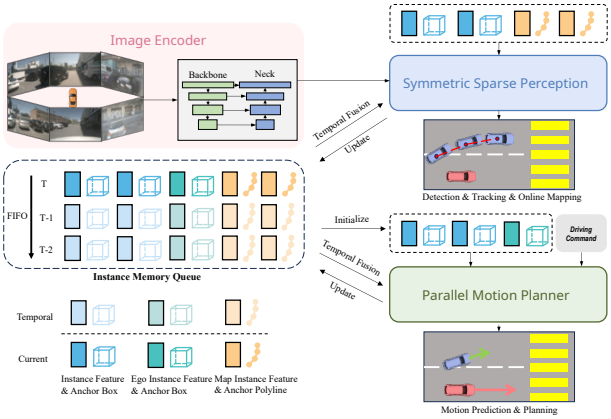


Fig. 2: Overview of SparseDrive. SparseDrive first encodes multi-view images into feature maps, then learns sparse scene representation through symmetric sparse perception, and finally perform motion prediction and planning in a parallel manner. An instance memory queue is devised for temporal modeling.

B. End-to-End Planning

The research of end-to-end planning has been ongoing since last century[21]. Early works[22], [23], [24] omit intermediate tasks like perception and motion prediction, which lack interpretability and are difficult to optimize. Some works[25], [26], [27], [28] construct explicit cost map from perception or prediction results to enhance interpretability, but rely on hand-crafted rules to select the best trajectory with minimum cost. Recently, UniAD[1] proposes a unified query design to integrate various tasks into a goal-oriented model, achieving remarkable performance in perception, prediction and planning. VAD[2] employs vectorized representation for scene learning and planning constraints. GraphAD[29] utilizes graph model for complex interactions in traffic scenes. FusionAD[30] extends end-to-end driving to multi-sensor input. However, previous methods mainly focus on scene learning, and adopt a straightforward design for prediction and planning, without fully considering the similarity between these two tasks, greatly limiting the performance.

III. Method

A. Overview

The overall framework of SparseDrive is depicted in Fig. 2. Specifically, SparseDrive is consisted of three parts:

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.

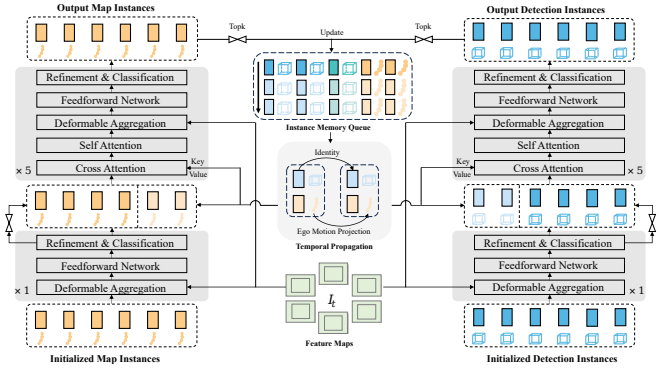


Fig. 3: Model architecture of symmetric sparse perception, which unifies detection, tracking and online mapping in a symmetric structure.

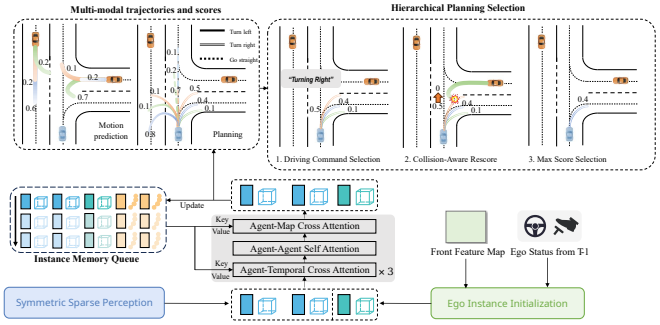


Fig. 4: Model structure of parallel motion planner, which performs motion prediction and planning simultaneously and outputs safe planning trajectory.

image encoder, symmetric sparse perception and parallel motion planner. Given multi-view images, the image encoder, including a backbone network and a neck, first encodes images to multi-view multi-scale feature maps $I = \left\{I_s \in \mathbb{R}^{N \times C \times H_s \times W_s} | 1 \leq s \leq S\right\}$, where S is the number of scales and N is the number of camera views. In symmetric sparse perception module, the feature maps I are aggregated into two groups of instances to learn the sparse representation of the driving scene. These two groups of instances, representing surrounding agents and map elements respectively, are fed into parallel motion planner to interact with an initialized ego instance. The motion planner predicts multi-modal trajectories of surrounding agents and ego vehicle simultaneously, and selects a safe trajectory as the final planning result through hierarchical planning selection strategy.

B. Symmetric Sparse Perception

As shown in Fig. 3, the model structure of sparse perception module exhibits a structural symmetry, unifying detection, tracking and online mapping together.

a) Sparse Detection: Surrounding agents are represented by a group of instance features $F_d \in \mathbb{R}^{N_d \times C}$ and anchor boxes $B_d \in \mathbb{R}^{N_d \times 11}$, where N_d is the number of anchors and C is the feature channel dimension. Each anchor box is formatted with location, dimension, yaw angle and velocity:

$$\{x, y, z, \ln w, \ln h, \ln l, \sin yaw, \cos yaw, vx, vy, vz\}.$$

The sparse detection branch consists of N_{dec} decoders, including a single non-temporal decoder and $N_{dec} - 1$ temporal decoders. Each decoder takes feature maps I, instance features F_d and anchor boxes B_d as input, outputs updated instance features and refined anchor boxes. The non-temporal decoder takes randomly initialized instances as input, while the inputs for temporal decoder come from both current frame and historical frame. Specifically, the non-temporal decoder includes three sub-modules: deformable aggregation, feedforward network (FFN) and the output layer for refinement and classification. The deformable aggregation module generates fixed or learnable keypoints around the anchor boxes B_d and projects them to feature maps I for feature sampling. The instance features F_d are updated by summation with sampled features, and are responsible for predicting the classification scores and the offsets of anchor boxes in the output layer. The temporal decoders have two additional multi-head attention layers: the temporal cross-attention between temporal instances from last frame and current instances, and the self-attention among current instances. In multi-head attention layer, the anchor boxes are transformed into high-dimensional anchor embedding $E_d \in \mathbb{R}^{N_d \times C}$, and serve as the positional encoding.

b) Sparse Online Mapping: Online mapping branch shares the same model structure with detection branch except different instance definition. For static map element, the anchor is formulated as a polyline with N_p points:

$$\left\{x_0, y_0, x_1, y_1, ..., x_{N_p-1}, y_{N_p-1}\right\}.$$

Then all the map elements can be represented by map instance features $F_m \in \mathbb{R}^{N_m \times C}$ and anchor polylines $L_m \in \mathbb{R}^{N_m \times N_p \times 2}$, where N_m is the number of anchor polylines.

c) Sparse Tracking: For tracking, we follow the ID assignment process of Sparse4Dv3[16]: once the detection confidence of an instance surpasses a threshold T_{thresh}, it is locked onto a target and assigned with an ID, which remains unchanged throughout temporal propagation. This tracking strategy does not need any tracking constraints, resulting in an elegant and simple symmetric design for sparse perception module.

C. Parallel Motion Planner

As shown in Fig. 4, the parallel motion planner consists of three parts: ego instance initialization, spatial-temporal interactions and hierarchical planning selection.

a) Ego Instance Initialization: Similar to surrounding agents, ego vehicle is represented by ego instance feature $F_e \in \mathbb{R}^{1 \times C}$ and ego anchor box $B_e \in \mathbb{R}^{1 \times 11}$. While ego feature is typically randomly initialized in previous methods, we argue that the ego feature also requires rich semantic and geometric information for planning, similar to motion prediction. However, the instance features of surrounding agents are aggregated from image feature maps I, which is not feasible for ego vehicle, since ego vehicle is in blind area

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.

of cameras. Thus we use the smallest feature map of front camera to initialize the ego instance feature:

$$F_e = \text{AveragePool}(I_{front,S}) \tag{1}$$

There are two advantages in doing so: the smallest feature map has already encoded the semantic context of the driving scene, and the dense feature map serves as a complementary for sparse scene representation, in case there are some blacklist obstacles, which can not be detected in sparse perception.

For ego anchor B_e, the location, dimension and yaw angle can be naturally set, as we are aware of these information of ego vehicle. For velocity, directly initialized from ground truth velocity leads to ego status leakage, as illustrated in [31]. So we add an auxiliary task to decode current ego status ES_T, including velocity, acceleration, angular velocity and steering angle. At each frame, we use the predicted velocity from last frame as the initialization of ego anchor velocity.

b) Spatial-Temporal Interactions: To consider the high-level interaction between all road agents, we concatenate the ego instance with surrounding agents to get agent-level instances:

$$F_a = \text{Concat}(F_d, F_e), B_a = \text{Concat}(B_d, B_e) \tag{2}$$

As the ego instance is initialized without temporal cues, which is important for planning, we devise an instance memory queue with the size of $(N_d + 1) \times H$ for temporal modeling, H is the number of stored frames. Then three types of interactions are performed to aggregate spatial-temporal context: agent-temporal cross-attention, agent-agent self-attention and agent-map cross-attention. Note that in temporal cross-attention of sparse perception module, the instances of current frame interact with all temporal instances, which we name as scene-level interaction. While for agent-temporal cross-attention here, we adopt instance-level interaction to make each instance focus on history information of itself.

Then, we predict the multi-modal trajectories $\tau_m \in \mathbb{R}^{N_d \times \mathcal{K}_m \times T_m \times 2}$, $\tau_p \in \mathbb{R}^{N_c \times \mathcal{K}_p \times T_p \times 2}$ and scores $s_m \in \mathbb{R}^{N_d \times \mathcal{K}_m}$, $s_p \in \mathbb{R}^{N_{cmd} \times \mathcal{K}_p}$ for both surrounding agents and ego vehicle, $\mathcal{K}_m$ and $\mathcal{K}_p$ are the number of modes for motion prediction and planning, T_m and T_p are the number of future timestamps for motion prediction and planning, and N_{cmd} is the number of driving command for planning. For planning, we additionally predict current ego status from ego instance feature.

c) Hierarchical Planning Selection: Now we have the multi-modal planning trajectory proposals, to select one safe trajectory τ_p^* to follow, we design a hierarchical planning selection strategy. First, we select a subset of trajectory proposals $\tau_{p,cmd} \in \mathcal{K}_p \times T_p \times 2$, corresponding to the high-level command cmd. Then, a novel collision-aware rescore module is adopted to ensure safety. With the motion prediction results, we can assess the collision risk of each planning trajectory proposal, for the trajectory with high collision probability, we reduce the score of this trajectory. In practice, we simply set the score of collided trajectory to 0. Finally, we select the trajectory with the highest score as the final planning output.

TABLE I: Training hyper-parameters.

Model	Training stage	Batch Size	Epochs	Lr	Backbone lr scale	Weight decay
SparseDrive-S	stage-1	8	100	4×10^{-4}	0.5	1×10^{-3}
SparseDrive-S	stage-2	6	10	3×10^{-4}	0.1	1×10^{-3}
SparseDrive-B	stage-1	4	80	3×10^{-4}	0.1	1×10^{-3}
SparseDrive-B	stage-2	4	10	3×10^{-4}	0.1	1×10^{-3}

D. End-to-End Learning

a) Multi-stage Training: The training of SparseDrive is divided into two stages. In stage-1, we train symmetric sparse perception module from scratch to learn the sparse scene representation. In stage-2, sparse perception module and parallel motion planner are trained together with no model weights frozen, fully enjoying the benefit of end-to-end optimization.

b) Loss Functions: The loss functions include the loss of four tasks, and the loss of each task can be further divider into classification loss and regression loss. For multi-modal motion prediction and planning task, we adopt the winner-takes-all strategy. For planning, there is an additional regression loss for ego status. We also introduce depth estimation as an auxiliary task to enhance the training stability of the perception module. The overall loss function for end-to-end training is:

$$\mathcal{L} = \mathcal{L}_{det} + \mathcal{L}_{map} + \mathcal{L}_{motion} + \mathcal{L}_{plan} + \mathcal{L}_{depth}. \tag{3}$$

IV. Experiments

Our experiments are conducted on open-loop nuScenes[32] and close-loop Bench2Drive-base[33] dataset, both containing 1000 complex driving scenes. Evaluation metrics mainly follows standard protocols[32], [33] and previous practice[1], [2], but we re-implement collision rate metric for more accurate evaluation[34], [31]. We have two variants of the model, which only differ in backbone network and input image resolution. For our small model SparseDrive-S, we use ResNet50[35] as backbone network and the input image size is 256×704. For our base model, SparseDrive-B, we change backbone network to ResNet101 and input image size to 512×1408. All experiments are conducted on 8 NVIDIA RTX 4090 24GB GPUs. We use AdamW optimizer[36] and Cosine Annealing scheduler[37] for model training. The training hyper-parameters are shown in Tab. I.

TABLE II: 3D detection and multi-object tracking results on nuScenes val dataset.

Method	NDS↑	mAP↑	AMOTA↑	AMOTP↓	Recall↑	IDS↓
ViP3D [38]	-	-	0.217	1.625	0.363	-
QD3DT [39]	-	-	0.242	1.518	0.399	-
MUTR3D [17]	-	-	0.294	1.498	0.427	3822
UniAD[1]	0.498	0.380	0.359	1.320	0.467	906
SparseDrive-S	0.525	0.418	0.386	1.254	0.499	886
SparseDrive-B	**0.588**	**0.496**	**0.501**	**1.085**	**0.601**	**632**

A. Main Results

We compare with prior state-of-the-arts, both modularized and end-to-end methods. On open-loop benchmark, our

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Online mapping results on nuScenes val dataset. †:Reproduced with official checkpoint.

Method	AP_{ped} ↑	$AP_{divider}$ ↑	$AP_{boundry}$ ↑	mAP↑
HDMapNet [18]	14.4	21.7	33.0	23.0
VectorMapNet [19]	36.1	47.3	39.3	40.9
MapTR [20]	**56.2**	**59.8**	**60.1**	**58.7**
VAD† [2]	40.6	51.5	50.6	47.6
SparseDrive-S	49.9	**57.0**	58.4	55.1
SparseDrive-B	**53.2**	56.3	**59.1**	**56.2**

TABLE IV: Motion prediction results on nuScenes val dataset.

Method	minADE(m)↓	minFDE(m)↓	MR↓	EPA↑
Cons Pos. [1]	5.80	10.27	0.347	-
Cons Vel. [1]	2.13	4.01	0.318	-
Traditional [38]	2.06	3.02	0.277	0.209
PnPNet [40]	1.15	1.95	0.226	0.222
ViP3D [38]	2.05	2.84	0.246	0.226
UniAD[1]	0.71	1.02	0.151	0.456
SparseDrive-S	0.62	0.99	0.136	0.482
SparseDrive-B	**0.60**	**0.96**	**0.132**	**0.555**

TABLE V: Planning results on nuScenes val dataset. *:LiDAR-based methods. †:Reproduced with official checkpoint.

Method	L2(m)↓				Col. Rate(%)↓			
	1s	2s	3s	Avg.	1s	2s	3s	Avg.
FF* [41]	0.55	1.20	2.54	1.43	0.06	0.17	1.07	0.43
EO* [42]	0.67	1.36	2.78	1.60	0.04	0.09	0.88	0.33
ST-P3 [25]	1.33	2.11	2.90	2.11	0.23	0.62	1.27	0.71
UniAD† [1]	0.45	0.70	1.04	0.73	0.62	0.58	0.63	0.61
VAD† [2]	0.41	0.70	1.05	0.72	0.03	0.19	0.43	0.21
SparseDrive-S	**0.29**	0.58	0.96	0.61	**0.01**	0.05	0.18	0.08
SparseDrive-B	**0.29**	**0.55**	**0.91**	**0.58**	**0.01**	**0.02**	**0.13**	**0.06**

lightweight model SparseDrive-S has surpassed previous SOTAs in all tasks, while our base model SparseDrive-B pushes the performance boundaries one step further. The main metrics for each task are marked in grey background in Tables.

a) Perception: For 3D detection in Tab. II, SparseDrive achieves **49.6%** mAP and **58.8%** NDS, yielding a significant improvement of **+11.6%** mAP and **+9.0%** NDS compared to UniAD[1]. For multi-object tracking in Tab. II, SparseDrive achieves **50.1%** AMOTA, and lowest ID switch of **632**, which surpasses UniAD[1] by **+14.2%** in terms of AMOTA and gets a **30.2%** reduction for ID switch, showing the temporal consistency of tracking tracklet. For online mapping in Tab. III, SparseDrive gets a mAP of **56.2%**, also surpassing previous end-to-end method VAD[2] by **+8.6%**.

b) Prediction: For motion prediction in Tab. IV, SparseDrive achieves the best performance with **0.60m** minADE, **0.96m** minFDE, **13.2%** MissRate and **0.555** EPA. Compared with UniAD[1], SparseDrive reduces errors by **15.5%** and **5.9%** on minADE and minFDE respectively.

c) Planning: For planning in Tab. V, among all methods, SparseDrive achieves a remarkable planning performance, with the lowest L2 error of **0.58m** and collision rate of **0.06%**. Compared with previous SOTA VAD[2], SparseDrive reduces L2 error by **19.4%** and collision rate by **71.4%**, demonstrating the effectiveness and safety of our method.

TABLE VI: Closed-loop results on Bench2Drive val dataset

Method	Closed-loop Metric			
	Driving Score ↑	Success Rate(%) ↑	Efficiency↑	Comfortness↑
AD-MLP [34]	18.05	0.00	48.45	22.63
UniAD-Tiny [1]	40.73	13.18	123.92	**47.04**
UniAD-Base [1]	45.81	16.36	129.21	43.58
VAD [2]	42.35	15.00	157.94	46.01
SparseDrive-S	**47.38**	**17.72**	**179.30**	28.80

TABLE VII: Multi-Ability Results on Bench2Drive val dataset.

Method	Ability (%) ↑					
	Merging	Overtaking	Emergency Brake	Give Way	Traffic Sign	Mean
AD-MLP [34]	0.00	0.00	0.00	0.00	0.00	0.00
UniAD-Tiny [1]	4.11	12.50	14.54	10.00	18.54	11.94
UniAD-Base [1]	9.46	12.50	20.00	30.00	23.03	19.00
VAD [2]	**12.50**	**17.50**	14.54	30.00	**25.55**	20.02
SparseDrive-S	10.66	7.50	**32.72**	**40.00**	22.22	**22.62**

d) Efficiency: As shown in Tab. VIII, besides the excellent performance, SparseDrive also achieves much higher efficiency for both training and inference. With the same backbone network, our base model achieves **4.8**× faster in training and **4.1**× faster in inference, compared with UniAD[1]. Our lightweight model can achieve **7.2** × and **5.0**× faster in training and inference.

e) Close-loop results: As shown in Tab. VI and VII, SparseDrive surpasses previous SOTA in close-loop evaluation, especially in safety-related ability(Emergency Brake and Give Way), indicating the safety of our method.

B. Ablation Study

We conduct extensive ablation studies to demonstrate the effectiveness of our design choices. We use SparseDrive-S as the default model for ablation experiments.

a) Effect of designs in Motion Planner: To underscore the significance of considering similarity between prediction and planning, we devised several specific experiments, as shown in Tab. IX. ID-2 ignores the impact of ego vehicle on surrounding agents by changing the parallel design for prediction and planning to sequential order, leading to worse performance for motion prediction and collision rate. ID-3 randomly initializes ego instance feature and set all parameters of ego anchor to 0. Removing the semantic and geometric information of ego instance leads to performance degradation in both L2 error and collision rate. ID-4 takes planning as a deterministic problem and only outputs one certain trajectory, resulting in highest collision rate. Moreover, ID-5 removes the instance-level agent-temporal cross-attention, seriously degrading the L2 error to 0.77m. For collision-aware rescore, we have detailed discussion in the following paragraph.

b) Collision-Aware Rescore: In previous methods[1], [29], a post-optimization strategy is adopted to ensure safety based on perception results. However, we argue that this strategy breaks the end-to-end paradigm, resulting in serious degradation in L2 error, as shown in Tab. X. Moreover,

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII: Efficiency comparison results. SparseDrive achieves high efficiency for both training and inference. Training time and FPS for UniAD are measured on 8 and 1 NVIDIA Tesla A100 GPUs respectively. Training time and FPS for SparseDrive are measured on 8 and 1 NVIDIA Geforce RTX 4090 GPUs respectively.

Method	Training Efficiency			Inference Efficiency			
	GPU Memory (G)	Batch Size	Time (h)	GPU Memory (M)	FLOPs (G)	Params (M)	FPS
UniAD [1]	50.0	1	48 + 96	2451	1709	125.0	1.8
SparseDrive-S	15.2	6	18 + 2	1294	192	85.9	9.0
SparseDrive-B	17.6	4	26 + 4	1437	787	104.7	7.3

TABLE IX: Ablation for designs in parallel motion planner. "PAL" means parallel design for motion prediction and planning task; "EII" means ego instance initialization; "MTM" means multiple mode for planning; "ATA" means agent-temporal cross-attention; "CAR" means collision-aware rescore.

ID	PAL	EII	MTM	ATA	CAR	Prediction			Planning L2(m)				Planning Coll.(%)			
						minADE	minFDE	MR	1s	2s	3s	Avg.	1s	2s	3s	Avg.
1	✓	✓	✓	✓	✓	0.623	**0.987**	0.136	**0.29**	**0.58**	0.96	**0.61**	**0.01**	**0.05**	**0.18**	**0.08**
2		✓	✓	✓	✓	0.641	1.008	0.138	0.30	0.58	0.95	**0.61**	0.02	0.06	0.23	0.10
3	✓		✓	✓	✓	**0.621**	0.988	**0.135**	0.31	0.60	0.98	0.63	0.03	0.07	0.21	0.11
4	✓	✓		✓	✓	0.626	1.002	0.136	0.33	0.66	1.08	0.69	0.03	0.11	0.60	0.25
5	✓	✓	✓		✓	0.634	1.003	0.138	0.40	0.74	1.16	0.77	0.02	0.13	0.32	0.16
6	✓	✓	✓	✓		0.623	**0.987**	0.136	**0.29**	**0.58**	**0.95**	**0.61**	**0.01**	0.06	0.30	0.12

TABLE X: Ablation for collision-aware rescore and post-optimization in UniAD[1].

Method	CAR	Post-optim.	Planning L2(m)				Planning Coll.(%)			
			1s	2s	3s	Avg.	1s	2s	3s	Avg.
UniAD[1]			0.32	0.58	0.94	0.61	0.15	0.24	0.36	0.25
UniAD[1]		✓	0.45	0.70	1.04	0.73	0.62	0.58	0.63	0.61
SparseDrive			**0.29**	**0.58**	**0.95**	**0.61**	**0.01**	0.06	0.30	0.12
SparseDrive	✓		**0.29**	**0.58**	0.96	**0.61**	**0.01**	**0.05**	**0.18**	**0.08**
SparseDrive		✓	0.44	0.73	1.11	0.76	0.29	0.21	0.38	0.30

TABLE XI: Ablation for planning mode.

Number of mode	Planning L2(m)				Planning Coll.(%)			
	1s	2s	3s	Avg.	1s	2s	3s	Avg.
1	0.33	0.66	1.08	0.69	0.03	0.11	0.60	0.25
2	0.33	0.65	1.08	0.69	0.01	0.12	0.42	0.18
3	0.30	0.59	0.97	0.62	**0.00**	0.08	0.43	0.17
6	**0.29**	**0.57**	**0.95**	**0.61**	0.01	**0.03**	**0.17**	**0.07**
9	0.33	0.63	1.04	0.66	0.01	0.09	0.36	0.15

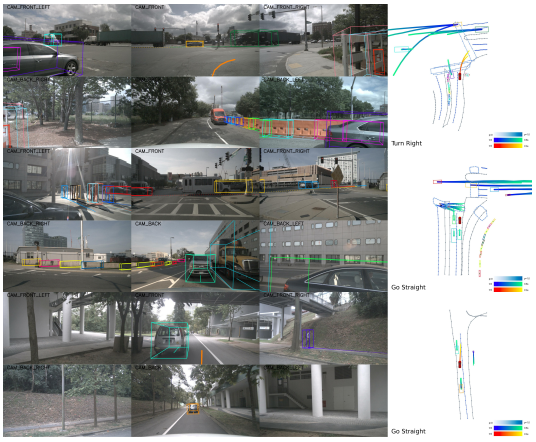


Fig. 5: Visualization results. SparseDrive learns to turn, yield to moving agents and avoid collision with obstacles.

under our re-implemented collision rate metric, the post-optimization does not make planning safer, but rather more dangerous. By contrast, our collision-aware rescore module reduces collision rate from 0.12% to 0.08%, with negligible increase in L2 error, showing the superiority of our method.

c) Multi-modal planning: We conduct experiments on the number of planning modes. As shown in Tab. XI, with the number of planning modes increases, the planning performance improves continuously until saturated at 6 modes, again proving the importance of multi-modal planning.

C. Qualitative Results

We visualize several difficult scenarios in Fig. 5.

V. Conclusion and Future Work

a) Conclusion.: In this work, we explore the sparse scene representation and review the task design in the realm of end-to-end autonomous driving. The resulting end-to-end paradigm SparseDrive achieves both remarkable performance and high efficiency. We hope the impressive performance of SparseDrive can inspire the community to rethink the task design for end-to-end autonomous driving and promote technological progress in this field.

b) Future work.: There still are some limitations in our work. First, the performance of our end-to-end model still falls behind the single-task method, for example, the online mapping task. Second, the scale of the dataset is not large enough to exploit the full potential of end-to-end driving.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang *et al.*, "Planning-oriented autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 853–17 862.

[2] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, "Vad: Vectorized scene representation for efficient autonomous driving," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 8340–8350.

[3] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16*. Springer, 2020, pp. 194–210.

[4] J. Huang, G. Huang, Z. Zhu, Y. Ye, and D. Du, "Bevdet: High-performance multi-camera 3d object detection in bird-eye-view," *arXiv preprint arXiv:2112.11790*, 2021.

[5] J. Huang and G. Huang, "Bevdet4d: Exploit temporal cues in multi-camera 3d object detection," *arXiv preprint arXiv:2203.17054*, 2022.

[6] Y. Li, Z. Ge, G. Yu, J. Yang, Z. Wang, Y. Shi, J. Sun, and Z. Li, "Bevdepth: Acquisition of reliable depth for multi-view 3d object detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 2, 2023, pp. 1477–1485.

[7] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. L. Rus, and S. Han, "Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *2023 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2023, pp. 2774–2781.

[8] J. Huang and G. Huang, "Bevpoolv2: A cutting-edge implementation of bevdet toward deployment," *arXiv preprint arXiv:2211.17111*, 2022.

[9] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai, "Bevformer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers," in *European conference on computer vision*. Springer, 2022, pp. 1–18.

[10] S. Chen, T. Cheng, X. Wang, W. Meng, Q. Zhang, and W. Liu, "Efficient and robust 2d-to-bev representation learning via geometry-guided kernel transformer," *arXiv preprint arXiv:2206.04584*, 2022.

[11] Y. Liu, T. Wang, X. Zhang, and J. Sun, "Petr: Position embedding transformation for multi-view 3d object detection," in *European Conference on Computer Vision*. Springer, 2022, pp. 531–548.

[12] Y. Liu, J. Yan, F. Jia, S. Li, A. Gao, T. Wang, and X. Zhang, "Petrv2: A unified framework for 3d perception from multi-camera images," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3262–3272.

[13] S. Wang, Y. Liu, T. Wang, Y. Li, and X. Zhang, "Exploring object-centric temporal modeling for efficient multi-view 3d object detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3621–3631.

[14] X. Lin, T. Lin, Z. Pei, L. Huang, and Z. Su, "Sparse4d: Multi-view 3d object detection with sparse spatial-temporal fusion," *arXiv preprint arXiv:2211.10581*, 2022.

[15] Lin, Xuewu and Lin, Tianwei and Pei, Zixiang and Huang, Lichao and Su, Zhizhong, "Sparse4d v2: Recurrent temporal fusion with sparse model," *arXiv preprint arXiv:2305.14018*, 2023.

[16] X. Lin, Z. Pei, T. Lin, L. Huang, and Z. Su, "Sparse4d v3: Advancing end-to-end 3d detection and tracking," *arXiv preprint arXiv:2311.11722*, 2023.

[17] T. Zhang, X. Chen, Y. Wang, Y. Wang, and H. Zhao, "Mutr3d: A multi-camera tracking framework via 3d-to-2d queries," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 4537–4546.

[18] Q. Li, Y. Wang, Y. Wang, and H. Zhao, "Hdmapnet: An online hd map construction and evaluation framework," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 4628–4634.

[19] Y. Liu, T. Yuan, Y. Wang, Y. Wang, and H. Zhao, "Vectormapnet: End-to-end vectorized hd map learning," in *International Conference on Machine Learning*. PMLR, 2023, pp. 22 352–22 369.

[20] B. Liao, S. Chen, X. Wang, T. Cheng, Q. Zhang, W. Liu, and C. Huang, "Maptr: Structured modeling and learning for online vectorized hd map construction," in *The Eleventh International Conference on Learning Representations*, 2022.

[21] D. A. Pomerleau, "Alvinn: An autonomous land vehicle in a neural network," *Advances in neural information processing systems*, vol. 1, 1988.

[22] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 4693–4700.

[23] F. Codevilla, E. Santana, A. M. López, and A. Gaidon, "Exploring the limitations of behavior cloning for autonomous driving," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9329–9338.

[24] A. Prakash, K. Chitta, and A. Geiger, "Multi-modal fusion transformer for end-to-end autonomous driving," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 7077–7087.

[25] S. Hu, L. Chen, P. Wu, H. Li, J. Yan, and D. Tao, "St-p3: End-to-end vision-based autonomous driving via spatial-temporal feature learning," in *European Conference on Computer Vision*. Springer, 2022, pp. 533–549.

[26] S. Casas, A. Sadat, and R. Urtasun, "Mp3: A unified model to map, perceive, predict and plan," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 14 403–14 412.

[27] A. Sadat, S. Casas, M. Ren, X. Wu, P. Dhawan, and R. Urtasun, "Perceive, predict, and plan: Safe motion planning through interpretable semantic representations," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXIII 16*. Springer, 2020, pp. 414–430.

[28] A. Cui, S. Casas, A. Sadat, R. Liao, and R. Urtasun, "Lookout: Diverse multi-future prediction and planning for self-driving," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 16 107–16 116.

[29] Y. Zhang, D. Qian, D. Li, Y. Pan, Y. Chen, Z. Liang, Z. Zhang, S. Zhang, H. Li, M. Fu *et al.*, "Graphad: Interaction scene graph for end-to-end autonomous driving," *arXiv preprint arXiv:2403.19098*, 2024.

[30] T. Ye, W. Jing, C. Hu, S. Huang, L. Gao, F. Li, J. Wang, K. Guo, W. Xiao, W. Mao *et al.*, "Fusionad: Multi-modality fusion for prediction and planning tasks of autonomous driving," *arXiv preprint arXiv:2308.01006*, 2023.

[31] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez, "Is ego status all you need for open-loop end-to-end autonomous driving?" *arXiv preprint arXiv:2312.03031*, 2023.

[32] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 11 621–11 631.

[33] X. Jia, Z. Yang, Q. Li, Z. Zhang, and J. Yan, "Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving," *arXiv preprint arXiv:2406.03877*, 2024.

[34] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuscenes," *arXiv preprint arXiv:2305.10430*, 2023.

[35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[36] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[37] Loshchilov, Ilya and Hutter, Frank, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.

[38] J. Gu, C. Hu, T. Zhang, X. Chen, Y. Wang, Y. Wang, and H. Zhao, "Vip3d: End-to-end visual trajectory prediction via 3d agent queries," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 5496–5506.

[39] H.-N. Hu, Y.-H. Yang, T. Fischer, T. Darrell, F. Yu, and M. Sun, "Monocular quasi-dense 3d object tracking," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 2, pp. 1992–2008, 2022.

[40] M. Liang, B. Yang, W. Zeng, Y. Chen, R. Hu, S. Casas, and R. Urtasun, "Pnpnet: End-to-end perception and prediction with tracking in the loop," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11 553–11 562.

[41] P. Hu, A. Huang, J. Dolan, D. Held, and D. Ramanan, "Safe local motion planning with self-supervised freespace forecasting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 12 732–12 741.

[42] T. Khurana, P. Hu, A. Dave, J. Ziglar, D. Held, and D. Ramanan, "Differentiable raycasting for self-supervised occupancy forecasting," in *European Conference on Computer Vision*. Springer, 2022, pp. 353–369.

Authorized licensed use limited to: TONGJI UNIVERSITY. Downloaded on October 19,2026 at 06:24:41 UTC from IEEE Xplore. Restrictions apply.